Intermediate DevOps
Day 2: Infrastructure as code and credentials

United Airlines · Pluralsight instructor-led training

Nick Lotz

27–29 October 2026

What you built on Day 1

- Harness Open Source running in Docker on your lab machine, with the `flightops` project and the `flight-status` repository.
- A CI pipeline in `.harness/ci.yaml` that lints the code and runs unit and API tests in containers.
- Integration tests that run against a Postgres background service.
- A branch rule on `main` that requires a pull request and a passing pipeline check.

Today you create the infrastructure the application will run on, and you manage the credentials it needs.

Day 2 agenda

Time (US Central)	Session
09:00	Day 1 recap
09:10	Lecture: Infrastructure as code concepts
09:40	Lecture: Terraform, CloudFormation and the AWS CDK
10:10	Break
10:20	Lab 04: Terraform foundations on Kubernetes (120 minutes)
12:20	Lunch
13:05	Lecture: Infrastructure as code in pipelines
13:35	Lab 05: Terraform in the pipeline (75 minutes)
14:50	Break
15:00	Lecture: Credential management
15:35	Lab 06: Managing credentials (75 minutes)
16:50	Summary and questions

How infrastructure was managed before code

- Engineers built and changed servers by hand, from a runbook or from memory.
- New servers came through tickets and took days or weeks.
- Servers that started the same became different over time. Fowler (2012) calls them snowflake servers.
- Rebuilding a failed server was slow, because nobody knew its full configuration.

Cloud APIs, starting with Amazon EC2 in 2006, made it possible to create servers and networks from a program in minutes.

What infrastructure as code means

Infrastructure as code applies software development practices to infrastructure (Morris, 2020).

- Definitions of servers, networks, clusters and permissions are files in version control.
- Every change is a commit with an author, a date and a reason.
- Changes are reviewed in pull requests before anyone applies them.
- Automation applies each change the same way, in every environment.

The files describe the infrastructure you intend to have. Lab 04 shows what happens when the real infrastructure differs.

Kinds of infrastructure as code tools

Category	What it manages	Examples
Provisioning	Cloud and platform resources, through their APIs	Terraform, OpenTofu, AWS CloudFormation, AWS CDK, Pulumi
Configuration management	Packages, files and services on existing servers	Ansible, Chef, Puppet, CFEngine
Image building	Machine images and container images	Packer, Dockerfiles
Kubernetes objects	Workloads and settings inside a cluster	YAML manifests, Helm, Kustomize

This course uses Terraform for provisioning today, and Kubernetes manifests on Day 3.

Imperative and declarative instructions

Imperative

You list the commands, in order. You decide what to do if a step already happened.

```
kubectl create namespace flightops-dev
kubectl create quota dev-quota \
  --hard=pods=10 -n flightops-dev
```

Declarative

You describe the result. The tool compares it with what exists and works out the steps.

```
resource "kubernetes_namespace_v1" "dev" {
  metadata {
    name = "flightops-dev"
  }
}
```

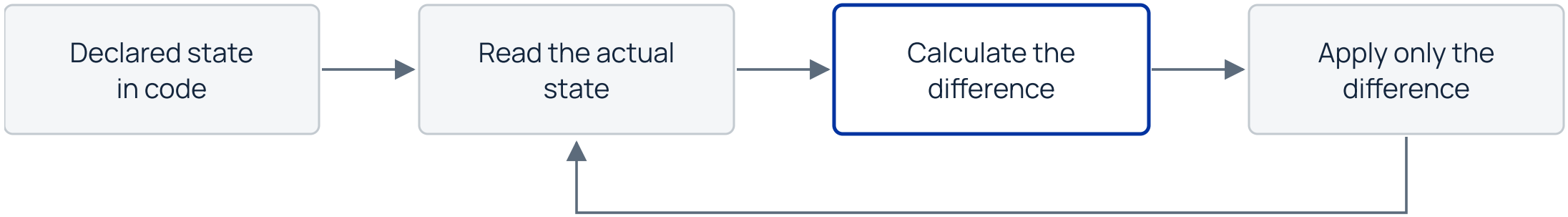
Idempotency

An operation is idempotent when running it many times has the same effect as running it once.

Command	First run	Second run
`kubectl create namespace dev`	Creates the namespace	Fails with AlreadyExists
`kubectl apply -f namespace.yaml`	Creates the namespace	Reports "unchanged"
`terraform apply`	Creates the namespace	Reports "No changes"
`mkdir -p /opt/app`	Creates the folder	Does nothing

You can re-run an idempotent operation after a failure or a timeout without side effects.

Desired state and convergence



Repeat on every run. If nothing differs, nothing changes.

Mark Burgess designed CFEngine in 1993 around this idea. Each run moves a system closer to the declared state. Terraform plans and Kubernetes controllers work the same way.

Configuration drift

- Drift is any difference between the code and the real infrastructure.
- It comes from manual changes in a console, emergency fixes, and other tools that change the same objects.
- Terraform reports drift in the next plan. CloudFormation has a drift detection command.
- After you find drift, apply the code to remove it, or change the code to keep it.

In Lab 04 you make a manual change with kubectl and find it in the next Terraform plan.

Mutable and immutable infrastructure

Mutable

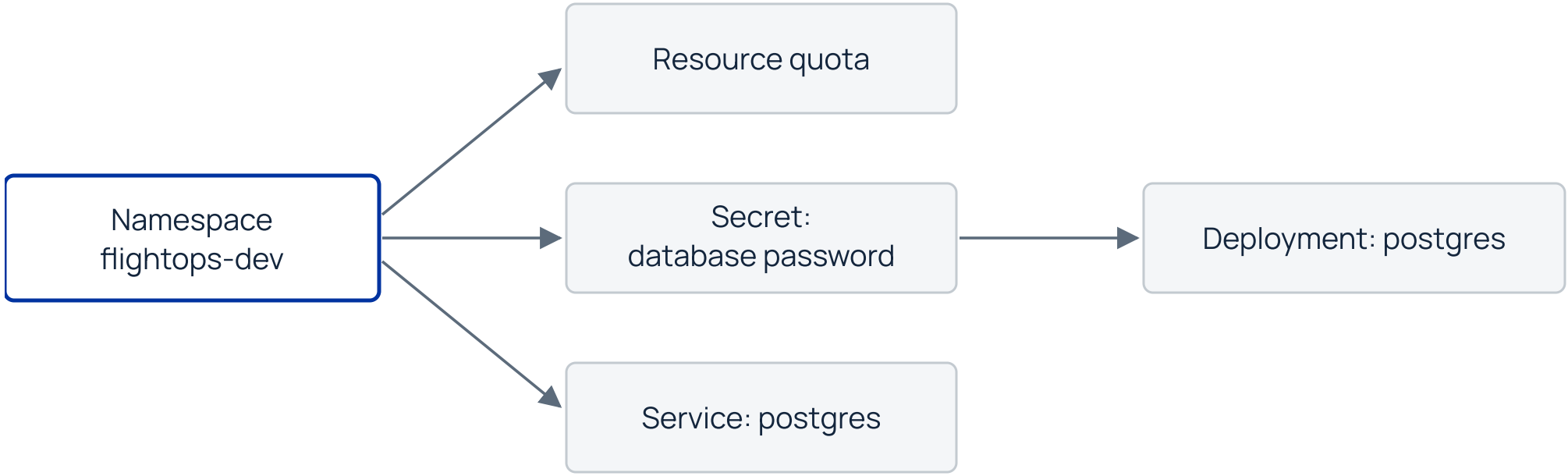
- Each server is updated in place.
- Its state depends on every change made since it was built.
- Rollback means undoing changes one by one.

Immutable

- A change produces a new image. New servers or containers replace the old ones.
- Nothing is modified after deployment.
- Rollback means deploying the previous image again.

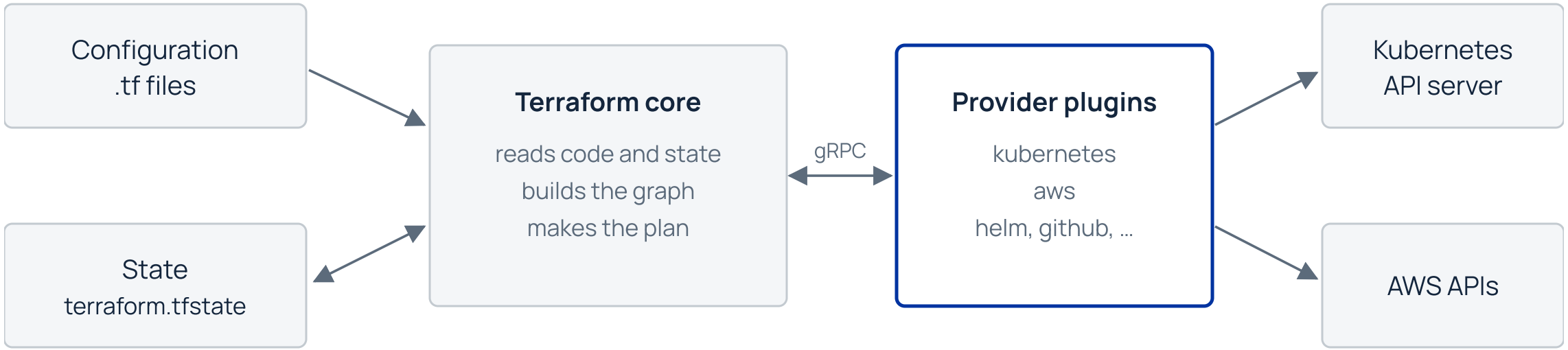
Containers and Kubernetes Deployments follow the immutable model. You will use it on Day 3.

The dependency graph



An arrow means the first object must exist before the second. Terraform builds this graph from references in the code. It creates independent objects in parallel, up to 10 at a time by default, and destroys in reverse order.

How Terraform is built



Terraform core has no knowledge of Kubernetes or AWS. Providers are separate programs that turn resources into API calls. `terraform init` downloads them from the Terraform Registry.

The Terraform language

```
variable "environments" {
  type    = set(string)
  default = ["dev", "prod"]
}

resource "kubernetes_namespace_v1" "env" {
  for_each = var.environments
  metadata {
    name = "flightops-${each.key}"
  }
}

output "namespaces" {
  value = keys(kubernetes_namespace_v1.env)
}
```

- `variable` is an input value.
- `resource` is an object Terraform manages. Its address is the type and the name.
- `for_each` makes one instance per item, such as `kubernetes_namespace_v1.env["dev"]`.
- `output` prints a value after apply and lets other code read it.

Provider versions and the lock file

```
terraform {
  required_version = ">= 1.17"
  required_providers {
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 3.4"
    }
  }
}
```

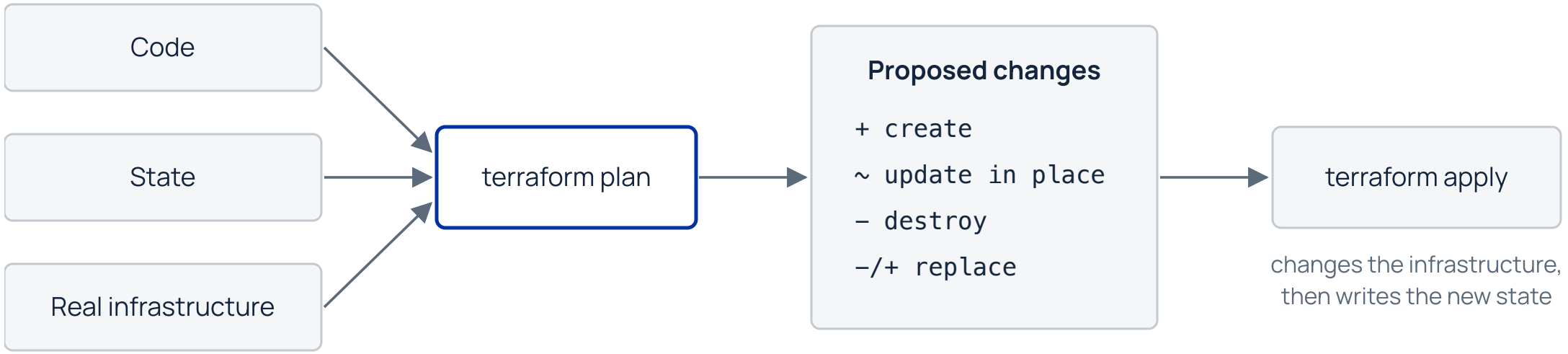
- `source` names the provider in the Terraform Registry.
- `~> 3.4` accepts version 3.4 or any later 3.x release. It rejects 4.0.
- `terraform init` records the selected version and its checksums in `.terraform.lock.hcl`. Commit that file.

Why Terraform keeps state

- State maps each resource address to a real object. `kubernetes_namespace_v1.env["dev"]` maps to the namespace `flightops-dev`.
- State records dependencies, so Terraform can delete objects in the right order after you remove them from the code.
- State stores the last known attribute values, which plans compare against.
- By default, state is a JSON file named `terraform.tfstate` in the working folder.

If the state is lost, Terraform no longer knows which objects it created.

Plan and apply



Nothing changes until apply runs. `terraform plan -out=tfplan` saves a plan, and `terraform apply tfplan` applies that saved plan without recalculating it.

Modules

```
module "postgres" {
  source   = "./modules/postgres"
  for_each = var.environments

  namespace = kubernetes_namespace_v1.env[each.key].metadata[0].name
}

# modules/postgres/variables.tf
variable "namespace" {
  type = string
}
```

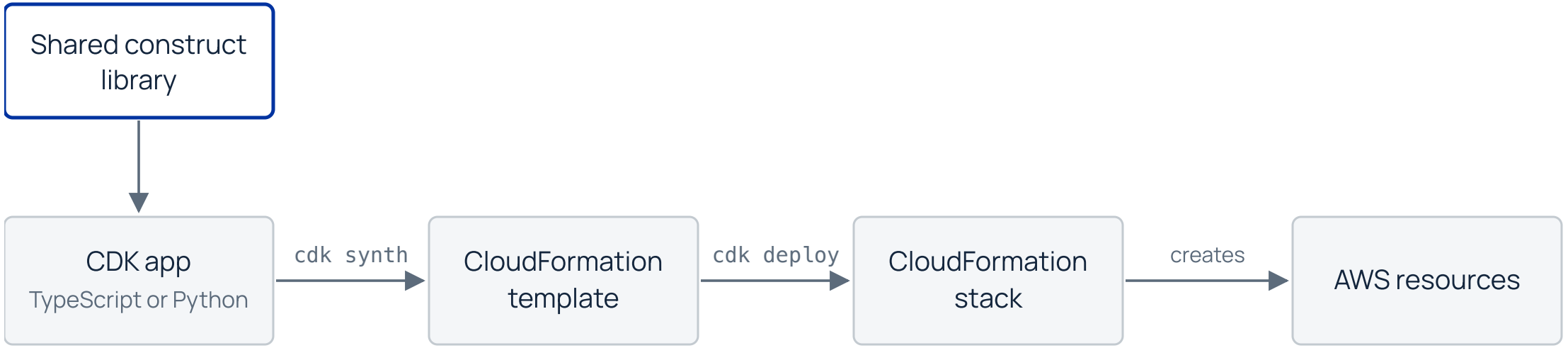
- A module is a folder of `.tf` files with input variables and outputs.
- The root module calls child modules and passes values to them.
- With `for_each`, one module builds the same database in dev and in prod.
- Modules can come from a local folder, a Git repository or a registry.

AWS CloudFormation

- You describe AWS resources in a JSON or YAML template.
- CloudFormation creates them as a stack and keeps the state inside AWS.
- A change set previews an update before you run it.
- If an update fails, CloudFormation rolls the stack back.
- Drift detection compares a stack with the real resources.

```
Resources:
  FlightDataBucket:
    Type: AWS::S3::Bucket
    Properties:
      VersioningConfiguration:
        Status: Enabled
```

The AWS Cloud Development Kit



CDK code produces a CloudFormation template, and CloudFormation creates the resources. Constructs are reusable classes that bundle resources with defaults. At United, teams use a shared CDK library. A library like this packages approved resource settings as constructs.

Terraform, CloudFormation and the CDK compared

	Terraform	CloudFormation	AWS CDK
Language	HCL	JSON or YAML	TypeScript, Python, Java, C#, Go
State	A state file in a backend you choose	Kept by AWS in the stack	Kept by AWS, through CloudFormation
Preview	`terraform plan`	Change set	`cdk diff`
Targets	Any API with a provider, such as AWS, Kubernetes or GitHub	AWS	AWS
Reuse	Modules	Nested stacks and modules	Constructs

Declared state, previews, review in pull requests and automated apply work the same way in all three.

Lab 04: Terraform foundations on Kubernetes

120 minutes. Open `labs/04-terraform-foundations.md` in the course repository.

- Install k3d, kubectl and Terraform, and create the local Kubernetes cluster.
- Write Terraform for dev and prod namespaces with quotas and default limits.
- Run plan and apply, make a manual change, and find the drift.
- Add a Postgres database per environment with a module, and push the code to Harness.

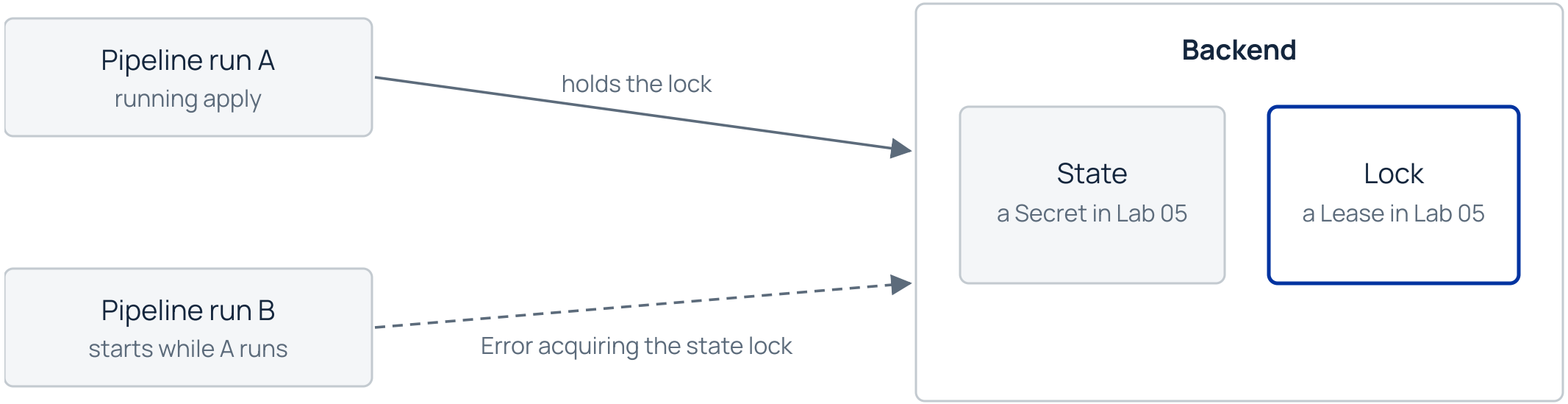
Why apply infrastructure code from a pipeline

When engineers apply from their laptops:

- Each laptop can have a different Terraform or provider version.
- State files are kept on laptops, or two people apply at the same time.
- Cloud credentials are copied to many machines.
- Nobody reviews the change, and there is no record of who applied it.

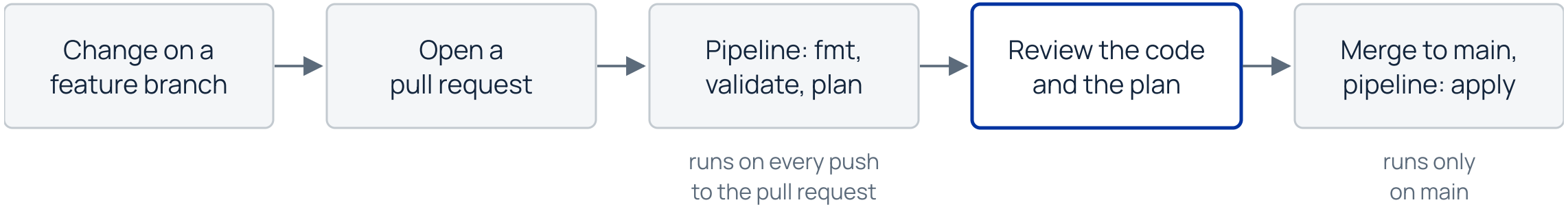
A pipeline runs one pinned version, uses one remote state, holds the credentials in one place, and applies only after review.

Remote state and locking



A backend stores state outside the working folder, for example in Amazon S3, HCP Terraform or a Kubernetes Secret. Locking stops two runs from changing the same state at once.

Plan on pull request, apply on merge



The branch rule from Lab 03 also protects `main` in the infrastructure repository. Reviewers approve the code change and its plan output together.

Reviewing a plan

- Read the summary first, for example "Plan: 1 to add, 2 to change, 0 to destroy."
- Look for destroy (`-`) and replace (`-/+`) actions. Replacing a database resource can delete its data.
- Check that the plan changes only the resources the pull request meant to change.
- A saved plan is valid only for the state it was made from. If the state changes first, Terraform rejects the plan as stale.

State is sensitive data

- State stores every attribute of every resource in plain text, including generated passwords and tokens.
- `sensitive = true` hides a value in plan output. The value is still written to state.
- Anyone who can read the state can read those values.
- Protect the backend with encryption at rest, and give access only to the pipeline and a few administrators.

In Lab 05 the state is stored in a Kubernetes Secret. In Lab 06 you find a database password in it.

Policy as code

A policy is a rule written as code and checked automatically. This Open Policy Agent rule, written in Rego, rejects a plan that deletes a namespace.

```
package terraform.namespaces

deny contains msg if {
  some rc in input.resource_changes
  rc.type == "kubernetes_namespace_v1"
  "delete" in rc.change.actions
  msg := sprintf("%s would be deleted", [rc.address])
}
```

The input is the plan as JSON, from `terraform show -json tfplan`. Other policy engines include HashiCorp Sentinel and AWS CloudFormation Guard. The Harness platform uses OPA policies to check pipelines and other resources.

GitOps principles

Principle	Meaning
Declarative	The desired state of the system is described declaratively.
Versioned and immutable	The desired state is stored with a complete version history.
Pulled automatically	Software agents pull the desired state from the source.
Continuously reconciled	Agents compare the actual state with the desired state and correct differences.

Lab 05 uses a push model, where the pipeline applies changes. Pull-based tools such as Argo CD and Flux run inside the cluster. Harness offers a GitOps module based on Argo CD.

Detecting drift from a pipeline

- Run a plan on a schedule, for example every night.
- With `-detailed-exitcode`, plan exits with 0 for no changes, 2 for changes and 1 for an error.
- On exit code 2, alert the team. Decide whether to revert the change or update the code.

```
rc=0
terraform plan -detailed-exitcode -input=false || rc=$?
if [ "$rc" -eq 2 ]; then echo "Drift found"; exit 1; fi
exit "$rc"
```

The `|| rc=$?` keeps the step's `set -e` shell from stopping before the check.

Lab 05: Terraform in the pipeline

75 minutes. Open `labs/05-terraform-pipeline.md` in the course repository.

- Move the Terraform state to the Kubernetes backend.
- Store a cluster credential for the pipeline as a Harness secret.
- Build a pipeline that runs fmt, validate and plan on pull requests, and apply on `main`.
- Change an environment through a pull request and watch the apply.

What counts as a credential

A credential is any value that proves an identity or grants access.

- Passwords and database connection strings
- API tokens, such as your Harness personal access token
- SSH and TLS private keys
- Cloud access keys, such as an AWS access key ID and secret access key
- kubeconfig files and Kubernetes ServiceAccount tokens

For each credential, know what it grants access to, who holds a copy, and when it expires.

Where secrets leak

Place	Example
Source code and Git history	A password committed and later deleted is still in the history.
Pipeline logs	`echo $TOKEN` or `set -x` prints the value.
Container images	A file copied in one layer stays in the image after a later layer deletes it.
Infrastructure state	Terraform state stores generated passwords in plain text.
Shell history and process data	Passwords typed as arguments; `docker inspect` shows environment variables.
Chat, tickets and wikis	A value pasted to help a colleague.

The secret-zero problem



Credential A is the secret zero.

A pipeline needs a credential to read from a secret store. If that first credential is a static token, it can leak like any other secret. Workload identity removes it. The platform proves who the workload is, for example with an AWS IAM role for an EKS pod or a Kubernetes ServiceAccount token.

Least privilege

"Every program and every user of the system should operate using the least set of privileges necessary to complete the job" (Saltzer and Schroeder, 1975).

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: flightops-dev
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "patch", "update"]
```

- Allow only the verbs the job needs.
- Limit the namespace and the resource types.
- Give each pipeline its own identity.
- Prefer credentials that expire.

Rotation and revocation



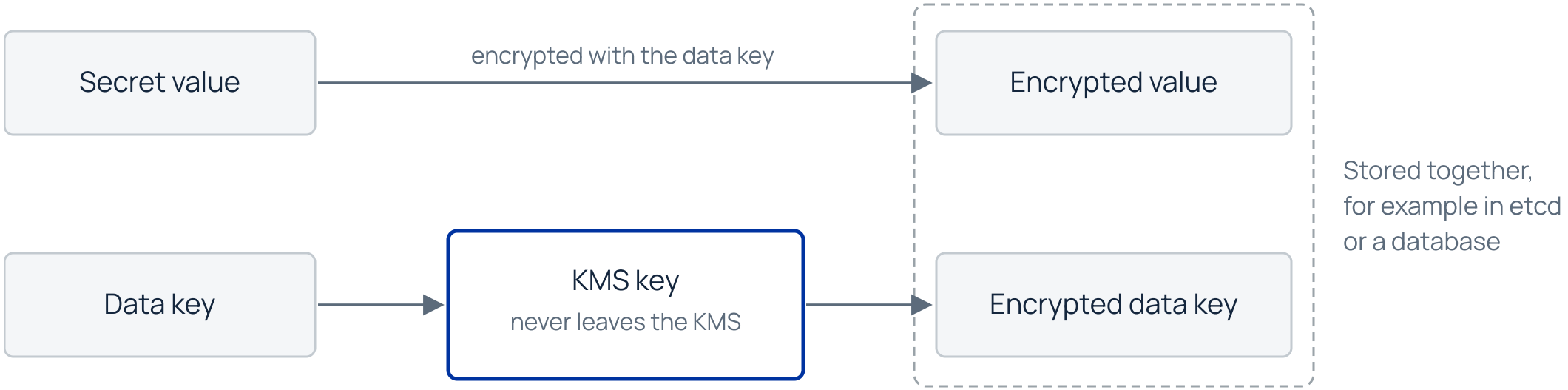
Rotation replaces a credential with a new value. Revocation makes the old value stop working. Rotate on a schedule, and at once after a suspected leak. Update every consumer before you revoke the old value.

Short-lived and dynamic credentials

Mechanism	Lifetime	Example
Kubernetes bound ServiceAccount token	Chosen per request, for example one hour	`kubectl create token deployer --duration=1h`
AWS STS temporary credentials	15 minutes to 12 hours	An IAM role assumed by a pipeline or a pod
Vault dynamic database credentials	A lease, for example one hour	A new database user per request, removed when the lease ends

A leaked short-lived credential stops working on its own. A long-lived credential works until someone revokes it.

Encryption at rest and envelope encryption



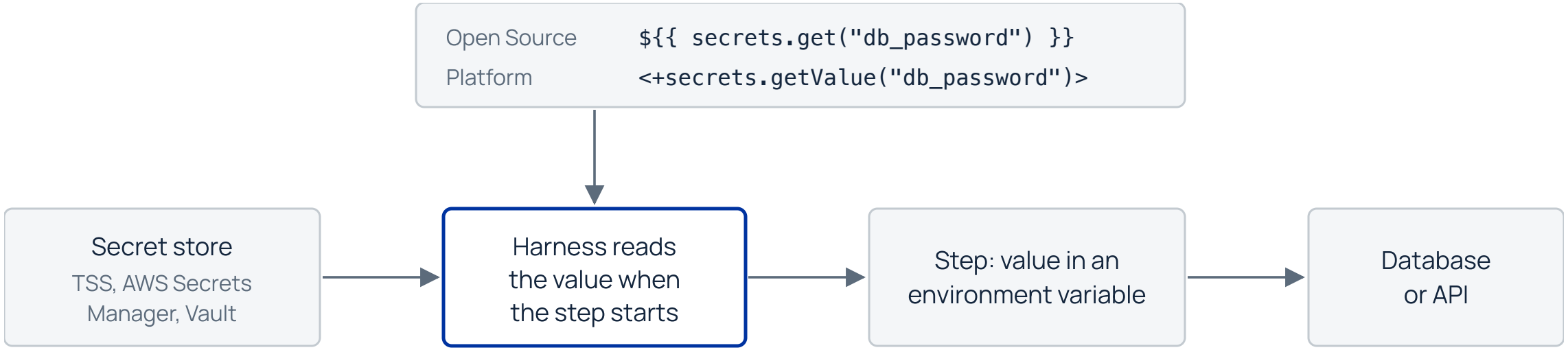
A data key encrypts the value. A key in a key management service (KMS), such as AWS KMS, encrypts the data key. The KMS key never leaves the KMS, and rotating it does not require re-encrypting the stored data.

Kubernetes Secrets

```
$ kubectl get secret db -n flightops-dev -o jsonpath='{.data.password}'
UzNjcjN0LVZhbHVl
$ echo UzNjcjN0LVZhbHVl | base64 -d
S3cr3t-Value
```

- The API server stores Secrets in etcd, base64-encoded. Anyone can decode base64.
- Encryption at rest is a separate API server setting. Amazon EKS can encrypt Secrets with a key in AWS KMS.
- Anyone allowed to read Secrets in a namespace, or to create Pods there, can read the values.

Secret stores and pipelines



The pipeline YAML holds a reference to the secret. At United, the enterprise store is TSS. The Harness platform reads external stores through secret manager connectors. Harness Open Source keeps secrets in its own database.

Secret scanning and log masking

Secret scanning

- Looks for secret patterns, such as AWS access keys, in commits.
- A pre-commit hook, such as gitleaks, blocks the commit on a laptop.
- Harness Open Source can reject a push and name the file and line.
- Custom secret formats can pass, because scanning relies on patterns.

Log masking

- Replaces known secret values with asterisks in logs.
- The Harness platform masks secrets. Harness Open Source 3.3.0 does not.
- A changed value, such as base64 or part of a string, is not masked.

Do not print secrets in logs. Scanning and masking only catch some mistakes.

Lab 06: Managing credentials

75 minutes. Open `labs/06-credentials.md` in the course repository.

- Find credentials on disk and in Terraform state, and decode a Kubernetes Secret.
- Store a secret in Harness and see that Harness Open Source does not mask it in logs.
- Create a least-privilege deploy ServiceAccount and test its permissions.
- Block a leaked key with secret scanning, then rotate a token.

Day 2 summary

- Infrastructure as code describes the desired state in version-controlled files and applies it with automation.
- Terraform compares the code, the state and the real infrastructure, shows a plan, and applies it.
- In a team, state is remote and locked, plans are reviewed in pull requests, and apply runs after merge.
- Credentials have a narrow scope, a short lifetime where possible, a place in a secret store, and never appear in logs.

Day 3 covers containers, Kubernetes deployments and an end-to-end delivery pipeline.

References

- Burgess, M. (1995). Cfengine: a site configuration engine. *USENIX Computing Systems* 8(3).
- Fowler, M. (2012). SnowflakeServer and PhoenixServer. martinfowler.com/bliki
- Fowler, C. (2013). Trash your servers and burn your code: immutable infrastructure and disposable components. chadfowler.com
- Morris, K. (2020). *Infrastructure as Code*, 2nd ed. O'Reilly.
- HashiCorp. Terraform documentation: state, backends, plan, modules. developer.hashicorp.com/terraform
- AWS. CloudFormation User Guide and AWS CDK Developer Guide. docs.aws.amazon.com
- OpenGitOps (2021). GitOps Principles v1.0.0. opengitops.dev
- Open Policy Agent documentation. openpolicyagent.org
- OWASP. Secrets Management Cheat Sheet. cheatsheetseries.owasp.org
- Saltzer, J. H., and Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE* 63(9).
- Kubernetes documentation: Secrets, encrypting data at rest, service account tokens, RBAC. kubernetes.io/docs
- AWS. IAM User Guide: temporary security credentials. docs.aws.amazon.com
- HashiCorp. Vault documentation: database secrets engine. developer.hashicorp.com/vault
- Harness documentation: secrets, secret managers, policy as code, open source pipelines. developer.harness.io